

Meeting Transcript Processor

Junior AI Engineer

Take-home

Build + explain decisions

BACKGROUND

KPI Media is a performance marketing agency. The team runs internal meetings daily, client briefs, campaign reviews, team syncs. After every meeting, someone is supposed to write up notes and share action items on Slack. In practice this rarely happens consistently. Notes are messy, action items get forgotten, and there's no single place to find what was decided.

Your task is to build an internal web tool that takes a raw meeting transcript, the kind exported from a recording tool like Fireflies or Otter and automatically structures it into something the team can actually use.

You're allowed and expected to use AI tools while building. We use them too. What we're evaluating is your judgment and the decisions you make, not whether you typed every line yourself.

WHAT TO BUILD

Core feature (Transcript processing)

A user opens the tool, either pastes a raw transcript into a text area or uploads a .txt file, gives the meeting a title, and hits process. The tool sends the transcript to an LLM and returns at minimum:

Summary	• 3 to 5 sentences covering what the meeting was about and what was decided • Not a play-by-play — the key points someone who wasn't in the meeting needs to know
Action items	• A list of tasks that came out of the meeting • Each item should have: the task itself, the owner (extracted from the transcript if mentioned), and the deadline (if mentioned — leave blank if not) • There will usually be 3 to 8 action items per meeting
Key decisions	• A short list of things explicitly decided in the meeting, separate from action items • Example: "Decided to pause the Google campaign until creative is ready" • Usually 1 to 4 per meeting

Once results come back, the user should be able to edit any field before exporting, the AI won't always get names or deadlines right. Export should produce a clean, readable output in Markdown or PDF format.

Beyond these core requirements, the rest is up to you. How you present the results, what extra fields or features you think are useful, how the editing experience works — these are your decisions. We're curious to see what you add.

History

Each processed transcript should be saved. The tool should have a history screen showing past meetings: title, date processed, and a way to re-open the full result.

STACK & TECHNICAL REQUIREMENTS

Frontend

React. How you structure and style it is up to you.

Backend — your choice

There's no required approach. Pick whatever gets you to a working, deployed app. Some options:

Option A BaaS — Supabase only

No custom backend server. Use Supabase directly from the frontend for database, file storage, and LLM calls via Edge Functions.

- Simpler to set up, less infrastructure to manage
- LLM API calls should go through a Supabase Edge Function to keep the API key out of the frontend

Option B Custom backend

Build your own backend server. Recommended: Hono — lightweight, TypeScript-native, deploys directly to Cloudflare Workers. Any other JS framework works too.

Option C No-code backend

Use a no-code automation tool (n8n, Make.com, or similar) to handle the LLM call and data storage, triggered from your React frontend.

- Valid choice if you can justify it — explain the tradeoffs vs. a custom backend

LLM Observability

Every LLM call in your app must be logged. At minimum each log entry should capture:

- The input sent to the model (prompt or messages)
- The output returned
- Model name used
- Token count (input + output)
- Latency (how long the call took)

Where you store and display this is up to you — a DB table, a log file, a UI debug panel, a third-party tool like LangFuse. It just needs to exist and be visible somewhere. You'll include a screenshot or short demo in your submission.

Deployment

Deploy somewhere publicly accessible. Recommended options:

- Cloudflare — Workers + Pages (pairs naturally with Hono and D1)
- Vercel — works well for both frontend and serverless functions
- Any other platform is fine as long as we can access a live URL

AI

- Use OpenAI API or OpenRouter
- Keep the API key server-side only

WHAT TO SUBMIT

Send everything as a single message or doc to your hiring contact. Include:

- 1 Live URL — must be publicly accessible, not localhost
- 2 GitHub repo link
- 3 Your system prompt pasted in full — the actual prompt you used, not a summary of it
- 4 A screenshot or short screen recording showing your LLM logs — at minimum: input, output, model, token count, and latency
- 5 A short written note covering: what stack you chose and why, what you added beyond the core requirements, what you cut due to time and why, what you'd improve with more time

HOW WE REVIEW THIS

We review your submission before the interview, then use the interview to go deeper on your decisions — not to ask you to rebuild anything. We're not looking for the most impressive tech stack. We're looking for:

- Does it work end to end — paste or upload a transcript, get structured output, edit it, export it
- Do you understand what you built — can you explain every decision in the write-up and in the interview
- Did you scope sensibly — did you know what to cut when time was tight
- How you think about AI — your prompt structure, output format choices, and how you handle failure cases
- What you added beyond the core — this tells us how you think as a product engineer, not just an implementer